Reflect-VLA: Internalizing Reflective Guidance via Dual-Path On-Policy Self-Distillation for Autonomous Driving

Yuechen Luo[1]*, He Chong[1]*, Fang Li[2]*, Shaoqing Xu[2†]*, Ruiqi Zhang[1], Hexin Zhang[3], Ziying Song[4], Lei Yang[5], Zhi-Xin Yang[2✉], Fuxi Wen[1✉]

[1]Tsinghua University [2]University of Macau [3]University of Science and Technology of China
[4]Beijing Jiaotong University [5]Nanyang Technological University

Abstract

Reinforcement Learning (RL) substantially improves planning performance in autonomous-driving Vision-Language-Action (VLA) models. However, existing RL methods rely on scalar trajectory rewards that measure overall quality but fail to identify specific planning errors or provide actionable correction signals. In long-tail scenarios, errors in scene understanding, behavior selection, and trajectory execution are compressed into a single score. We observe that Reflective Guidance turns trajectory evaluation into explicit diagnoses and corrective instructions, substantially improving planning accuracy. Yet such guidance requires an initial prediction and its evaluation results, making it unavailable for direct single-pass inference. We propose **Reflect-VLA**, which treats Reflective Guidance as training-time privileged information and internalizes its planning benefits through **Dual-Path On-Policy Self-Distillation (Dual-OPSD)**. Unlike scalar-reward optimization, Dual-OPSD uses the driving score only to route student-generated trajectories. It distills dense token-level corrective distributions from the teacher for risky trajectories while applying self-imitation to preserve reliable ones. To provide reliable corrective distributions on student rollouts, the teacher is progressively constructed through Driving-Knowledge Pretraining, Planning-and-Refinement SFT, and Self-Refinement Rejection Sampling. At inference time, Reflect-VLA generates trajectories in a single pass without explicit guidance or iterative refinement. Experiments on NAVSIM v1/v2 and Bench2Drive demonstrate competitive planning performance while reducing training time by over 90% compared with RL.

1 Introduction

Autonomous driving is shifting from conventional modular pipelines toward end-to-end Vision-Language-Action (VLA) systems (Jiang et al. 2025b). Unlike modular systems (Hu et al. 2023; Jiang et al. 2023; Chen et al. 2024) that coordinate perception, prediction, and planning through handcrafted interfaces, VLA models integrate visual observations, language instructions and action generation within a unified architecture. This formulation enables joint perception, reasoning, and planning, and supports flexible decision making in complex driving environments. Recent driving VLA methods commonly adopt a multi-stage training paradigm (Li et al. 2025c; Zhou et al. 2025b; Luo et al. 2025, 2026b; Lu et al. 2026). They first acquire domain knowledge from large-scale driving question-answering data, then learn trajectory generation through supervised fine-tuning, and finally improve the policy using reinforcement learning or planning-oriented objectives.

Despite progress, VLA models struggle to learn from plan-

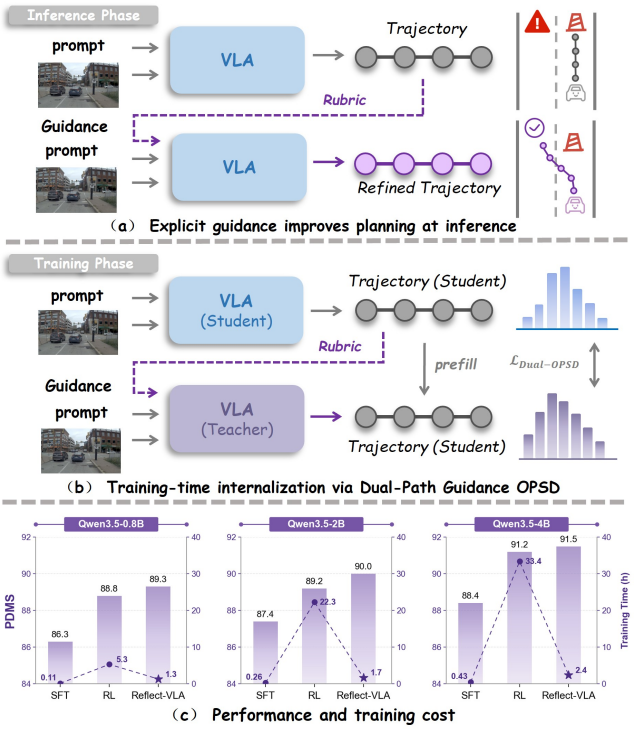


Figure 1: **From Reflective Guidance to Internalized Planning.** (a) Reflective Guidance corrects an unsafe initial trajectory during iterative inference, revealing substantial planning gains. (b) Reflect-VLA transfers these gains through Dual-OPSD by aligning teacher and student token distributions along student-generated trajectories. (c) Across different model scales, Reflect-VLA improves planning performance over SFT while requiring substantially less training time than RL, enabling guidance-free single-pass planning.

*These authors contributed equally.
†Corresponding author.

ning failures, especially in complex and long-tail scenarios. A scalar planning score can identify a poor trajectory but cannot reveal whether the error arises from scene understanding, behavior selection, or trajectory execution, nor indicate how to correct it. Trajectory-level rewards collapse these heterogeneous failure modes into a single value, leaving the model without a direction for improvement. ELF-VLA (Luo et al. 2026a) alleviates this problem by introducing structured failure information, showing that detailed feedback can guide the correction of failed trajectories. However, how to convert feedback-assisted correction into an intrinsic single-pass planning capability remains underexplored.

We begin with a key observation: **Reflective Guidance provides substantial planning gains**. As illustrated in Fig. 1, a VLA model may produce an incorrect trajectory during direct inference, yet generate a markedly better trajectory when provided with evaluation-derived diagnoses and corrective suggestions (Yang et al. 2026). This benefit cannot be obtained by simply treating Reflective Guidance as an additional deployment input. Constructing the guidance requires both an initial trajectory and its evaluation results, whereas an autonomous driving system must produce an executable plan before acting. The central challenge is therefore not to perform iterative refinement at inference time, but to transfer the planning preferences induced by Reflective Guidance into the model's direct trajectory-generation policy.

To this end, we propose **Reflect-VLA**, a training framework that treats Reflective Guidance as privileged information and internalizes its planning benefits through **Dual-Path On-Policy Self-Distillation (Dual-OPSD)**. Starting from the same VLA checkpoint, the model supplied with Reflective Guidance acts as the frozen teacher, while the model receiving standard inputs acts as the student. The student first generates trajectories from the input available at deployment. A planning score then routes each trajectory to one of two learning paths. For risky or suboptimal trajectories, the teacher provides dense token-level distributions that encode corrective planning preferences. For reliable trajectories, a self-imitation objective preserves the student's existing decisions. The scalar score is therefore used only for trajectory-quality-aware routing, while the teacher distribution supplies the fine-grained supervision required for correction.

Effective self-distillation requires a teacher that can refine trajectories drawn from the student's rollout distribution. We construct this teacher using a progressive strategy comprising Driving-Knowledge Pretraining, Planning-and-Refinement SFT, and Self-Refinement Rejection Sampling. The first two stages establish driving knowledge, direct trajectory generation, and guidance-assisted refinement. The final stage exposes the teacher to student-generated failures and retains verified refinements, reducing the distribution gap between offline refinement data and on-policy trajectories. After Dual-OPSD, Reflect-VLA performs single-pass trajectory generation without Reflective Guidance, an online planning evaluator, or an iterative correction procedure.

Our main contributions are summarized as follows:

- We construct Reflective Guidance that converts trajectory evaluation results into structured, trajectory-specific diagnoses and corrective suggestions, and use it as privileged information available only during training.
- We propose Dual-OPSD, a trajectory-quality-aware self-distillation method that transfers corrective distributions for risky trajectories while preserving reliable planning.
- We develop a progressive teacher-construction strategy that combines driving-knowledge pretraining, planning-and-refinement supervision, and self-refinement rejection sampling to support stable guidance internalization and efficient single-pass planning.

2 Related Work

2.1 VLA models for autonomous driving

VLMs and VLA models have been increasingly studied for autonomous driving, with applications ranging from scene understanding (Jiang et al. 2024, 2025a; Marcu et al. 2024) to direct trajectory prediction (Xing et al. 2025; Zhao et al. 2025; Liu et al. 2025; Fu et al. 2025). To improve interpretability and planning accuracy, several methods introduce explicit intermediate reasoning, such as Chain-of-Thought reasoning (Liu et al. 2025; Zhao et al. 2025). Their training pipelines typically begin with supervised fine-tuning on driving data and subsequently apply reinforcement learning, including Group Relative Policy Optimization, to improve planning performance (Luo et al. 2025; Zhou et al. 2025b). Although scalar planning rewards provide a direct measure of trajectory quality, they offer limited supervision when diverse failure causes receive similarly low scores. This issue is especially pronounced in long-tail scenarios, where repeated low-reward rollouts can make policy optimization inefficient. ELF-VLA (Luo et al. 2026a) addresses this limitation by using teacher-generated feedback to diagnose planning failures and construct improved training samples. Our work instead focuses on internalizing the benefit of such structured guidance so that it is no longer required during inference.

2.2 On-Policy Distillation and On-Policy Self-Distillation

On-policy distillation (OPD) and on-policy self-distillation (OPSD) are post-training approaches that supervise a student on trajectories sampled from its own policy (Agarwal et al. 2024; Zhao et al. 2026). Unlike off-policy distillation, this formulation queries a teacher with student-generated prefixes, thereby providing token-level supervision over the distribution encountered during training. OPD commonly relies on a separately trained teacher, whereas OPSD can derive the teacher from the same model by conditioning it on privileged information (Ye et al. 2026; Sang et al. 2026), such as ground-truth answers, additional instructions, or preference information. Recent work has combined dense token-level signals from OPD with reinforcement learning for VLA post-training (Zhong et al. 2026), but its application to autonomous-driving planning remains limited. We formulate Reflective Guidance as privileged information and use it to obtain corrective teacher distributions on student-generated trajectories. Dual-OPSD further accounts for trajectory quality by distilling corrections only when needed and preserving decisions that are already reliable.

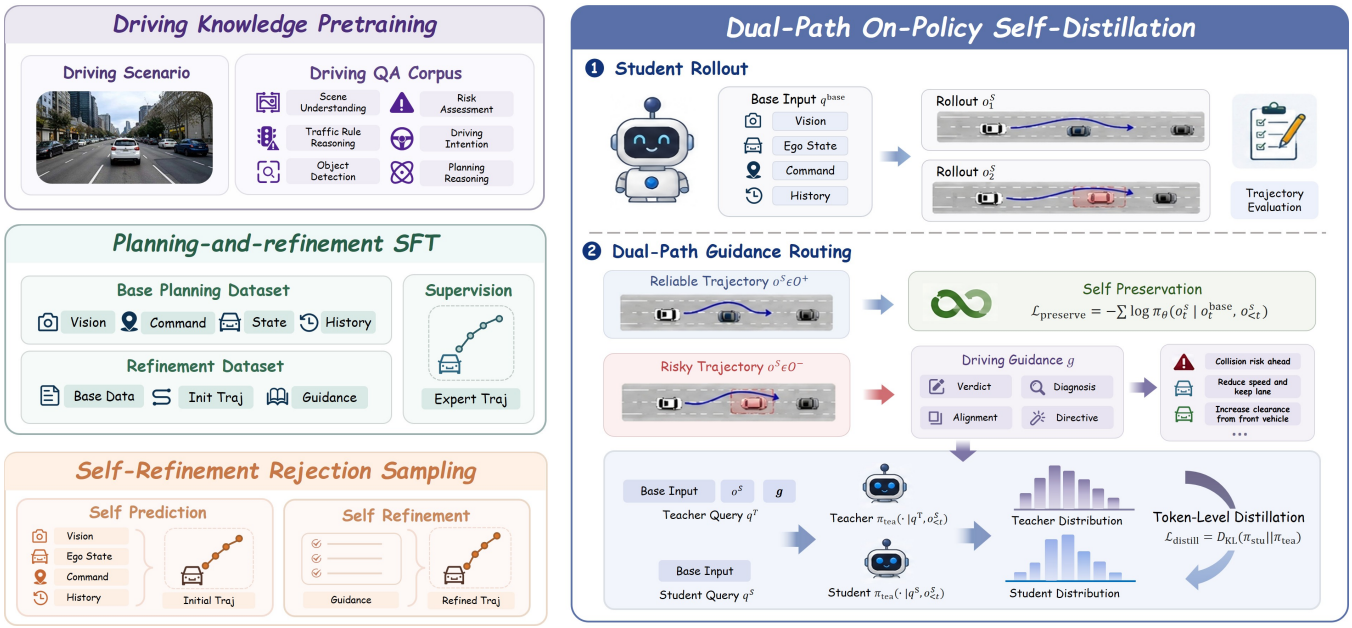


Figure 2: **Overview of Reflect-VLA.** Reflective Guidance is constructed from trajectory evaluation and used as privileged information to train a refinement-capable teacher. Dual-OPSD then transfers corrective distributions to the student on risky trajectories while preserving reliable student trajectories. Only the student is used for single-pass inference.

3 Method

Figure 2 illustrates the overall framework of our Reflect-VLA, which comprises a three-stage training pipeline that equips the model with trajectory prediction and refinement capabilities, followed by the proposed Dual-Path On-Policy Self-Distillation.

3.1 VLA Inputs Formulation

We formulate end-to-end driving planning as conditional autoregressive trajectory generation. The standard driving input is

$$q^{\text{base}} = (I_{\text{cam}}, q_{\text{cmd}}, q_{\text{ego}}, q_{\text{his}}), \tag{1}$$

where I_{cam} denotes the front-view camera observation, q_{cmd} is the high-level navigation command (e.g., *Go Straight*, *Turn Left*, and *Turn Right*), q_{ego} contains the ego-vehicle state (e.g., velocity and acceleration), and q_{his} represents the historical ego trajectory. Given q^{base}, the VLA autoregressively predicts future waypoints:

$$o = \{(x_t, y_t, h_t)\}_{t=1}^{T} \sim \pi_\theta(o \mid q^{\text{base}}), \tag{2}$$

where (x_t, y_t, h_t) denotes the position and heading at the t-th future timestep. To cast continuous trajectory prediction as autoregressive token generation, we convert the absolute waypoints into relative displacements $\{(\Delta x_t, \Delta y_t, \Delta h_t)\}_{t=1}^{T}$ and discretize these relative displacements into special trajectory tokens. Details are provided in Appendix D.

To improve the planner's reasoning ability during training, we augment the base input with Reflective Guidance, forming the guidance-conditioned input

$$q^{\text{gui}} = (q^{\text{base}}, o^{\text{init}}, g), \tag{3}$$

where o^{init} is the low-quality initial trajectory predicted under q^{base}, and g denotes the designed **Reflective Guidance Corpus**, which comprises four components: (1) Trajectory Verdict, summarizing the overall trajectory evaluation; (2) Planning Diagnosis, identifying the dominant causes of planning failures; (3) Expert Alignment, relating these failures to discrepancies from expert trajectory; and (4) Refinement Directive, suggesting high-level corrective actions. The Reflective Guidance is designed to expose the reasoning process behind trajectory refinement, thereby guiding the model to generate safer and higher-quality trajectories. Details of the guidance construction are provided in Appendix D.

During training, the model is conditioned on either q^{base} or q^{gui}. With q^{base}, it learns direct trajectory generation, whereas the additional reflective guidance in q^{gui} enables trajectory diagnosis and correction. Our goal is to distill the planning capability elicited by the guidance-conditioned input into direct trajectory generation through OPSD, enabling the model to retain strong planning and reasoning abilities even without guidance during inference:

$$o^* = \arg\max_o \mathcal{P}(o \mid q^{\text{base}}). \tag{4}$$

3.2 Data Construction

To develop both direct trajectory prediction and guidance-conditioned trajectory refinement capabilities, we first prepare the corresponding training data as follows.

Pretraining and Base Data. We first collect a diverse set of open-source driving QA datasets, including LingoQA (Marcu et al. 2024), DriveGPT4 (Xu et al. 2024), Senna (Jiang et al. 2024), DriveLM (Sima et al. 2024),

NuScenes-QA (Qian et al. 2024), NuInstruct (Ding et al. 2024), and DriveLMM (Ishaq et al. 2025). Pretraining on these datasets equips the general VLM with domain-specific knowledge for autonomous driving, including road semantics, traffic regulations, and driving-intent understanding.

We then construct the base planning dataset, with each sample containing a front-view camera image, a high-level navigation command, the ego-vehicle state, a historical trajectory, and a ground-truth future trajectory represented as discretized special trajectory tokens.

Guidance-conditioned Refinement Data. To enable the model to refine low-quality initial trajectories, we further construct a guidance-conditioned refinement dataset. Specifically, for each driving scenario, we first build a candidate trajectory set $\mathcal{O}_{\text{can}} = \mathcal{O}_{\text{plan}} \cup \mathcal{O}_{\text{rule}}$, where $\mathcal{O}_{\text{plan}}$ denotes the trajectories generated by a classical planner, while $\mathcal{O}_{\text{rule}}$ consists of trajectories obtained through rule-based perturbation. Each candidate is assessed using the NAVSIM evaluator (Dauner et al. 2024), together with map constraints and trajectory-level analysis. Based on the resulting scores, low-quality candidates are selected as the initial trajectories o^{init} for refinement. In particular, the NAVSIM evaluator provides the Predictive Driver Model Score (PDMS), which comprehensively measures collision safety, drivable-area compliance, time-to-collision, ego progress, and overall trajectory quality. The Reflective Guidance g is derived from the evaluation results, transforming scalar scores into structured, trajectory-specific corrective supervision. Consequently, training samples in the guidance-conditioned refinement dataset can be represented as:

$$\mathcal{D}_{\text{refine}} = \{(q^{\text{gui}}, o^{\text{ref}})\} = \{(q^{\text{base}}, o^{\text{init}}, g, o^{\text{ref}})\}. \quad (5)$$

where o^{ref} denotes the target refined trajectory, which can either be the ground-truth trajectory or a high-quality refined trajectory verified by the trajectory evaluator.

Self-Refinement Rejection Sampling. Training on the constructed planning and refinement data enables the model to leverage guidance for trajectory correction. However, because the initial trajectories in this dataset are generated offline, they may not fully capture the failure patterns encountered during model rollout. To mitigate this distribution mismatch, we introduce Self-Refinement Rejection Sampling. Specifically, the model first predicts an initial trajectory from the base input. When the trajectory is identified as problematic, the same model subsequently refines its prediction in response to the corresponding guidance. We retain only refined trajectories that satisfy the validity constraints and outperform their initial counterparts, and use these high-quality samples to further optimize the model.

3.3 Three-stage SFT for Planning and Refinement

We adopt a progressive three-stage training strategy consisting of driving-knowledge pretraining, planning-and-refinement supervised fine-tuning, and self-refinement rejection sampling. In the first stage, the model acquires essential domain knowledge for autonomous driving. In the second stage, it is trained on a mixture of base planning data and guidance-conditioned refinement data, enabling both direct trajectory generation and trajectory refinement. In the final stage, Self-refinement rejection sampling trains the model to correct failures from its own unguided rollouts, thereby yielding a more reliable teacher for subsequent on-policy self-distillation.

3.4 Dual-Path On-Policy Self-Distillation

After the three-stage training process, the model acquires the ability to refine trajectories with reflective guidance. Inspired by on-policy self-distillation for LLMs (Zhao et al. 2026), where a single model serves as both teacher and student under different input contexts, we propose **Dual-Path Online Policy Self-Distillation (Dual-OPSD)**, which distills the planning capability elicited by guidance-conditioned inputs into the model's direct trajectory generation behavior.

In Dual-OPSD, the same model serves as both the *student* and the *teacher* under different input contexts. With only the base input q^{base}, it acts as the student and performs on-policy rollout:

$$o^S \sim \pi_\theta(\cdot|q^{\text{base}}), \quad (6)$$

The generated trajectory is then evaluated by the NAVSIM evaluator. According to the resulting PDMS r, the rollout samples are divided into reliable trajectories $\mathcal{O}^+$ and risky trajectories $\mathcal{O}^-$ using a predefined threshold s:

$$o^S \in \begin{cases} \mathcal{O}^+, & r \geq s, \\ \mathcal{O}^-, & r < s, \end{cases} \quad (7)$$

This categorization enables two complementary optimization paths. For reliable trajectories in $\mathcal{O}^+$, we adopt a self-preservation objective to maintain the student's existing planning capability:

$$\mathcal{L}_{\text{preserve}} = -\sum_t \log \pi_\theta(o_t^S|q^{\text{base}}, o_{<t}^S), \quad (8)$$

where the generated trajectory serves as the self-supervised target. This prevents unnecessary modification of already valid behaviors during distillation.

For risky trajectories in $\mathcal{O}^-$, we construct the corresponding guidance g based on the student rollout and evaluation results. A frozen version of the same model serves as the teacher and receives the privileged input $q^T = (q^{\text{base}}, o^S, g)$, while the student remains restricted to q^{base}. The teacher's corrective planning preferences are transferred to the student by minimizing the token-level reverse KL divergence between their predictive distributions:

$$\mathcal{L}_{\text{distill}} = \sum_t D_{\text{KL}}\left(\pi_{\text{stu}}(\cdot|q^{\text{base}}, o_{<t}^S) \| \pi_{\text{tea}}(\cdot|q^T, o_{<t}^S)\right), \quad (9)$$

The overall Dual-OPSD objective combines the two pathways:

$$\mathcal{L}_{\text{Dual-OPSD}} = \mathbb{1}_{\{r \geq s\}} * \mathcal{L}_{\text{preserve}} + \mathbb{1}_{\{r < s\}} * \mathcal{L}_{\text{distill}}. \quad (10)$$

By adaptively preserving reliable behaviors and distilling corrective preferences on failed rollouts, Dual-OPSD enables the model to progressively internalize the reflective planning capability acquired by the guidance-conditioned teacher while maintaining guidance-free inference.

Method	Image	Lidar	NC↑	DAC↑	TTC↑	CF↑	EP↑	PDMS↑
End-to-End Methods								
UniAD (Hu et al. 2023)	✓		97.8	91.9	92.9	**100**	78.8	83.4
TransFuser (Chitta et al. 2022)	✓	✓	97.7	92.8	92.9	**100**	84.0	84.0
DiffusionDrive (Liao et al. 2025)	✓	✓	98.2	96.2	94.7	**100**	82.2	88.1
WoTE (Li et al. 2025b)	✓	✓	98.5	96.8	94.9	99.9	81.9	88.3
DVGT-2 (Zuo et al. 2026)	✓		98.7	97.9	95.8	**100**	84.3	90.3
Vision-Language-Action Methods								
AutoVLA-3B (Zhou et al. 2025b)	✓		98.4	95.6	**98.0**	99.9	81.9	89.1
AutoDrive-R^2-7B (Yuan et al. 2025)	✓		98.5	95.9	95.4	**100**	82.7	89.1
DriveVLA-W0-7B (Li et al. 2025a)	✓		98.7	**99.1**	94.9	99.3	83.3	90.2
AdaThinkDrive-8B (Luo et al. 2025)	✓		98.4	97.8	95.2	**100**	84.4	90.3
ReCogDrive-2B (Li et al. 2025c)	✓		97.9	97.3	94.6	**100**	**87.3**	90.8
ELF-VLA-8B (Luo et al. 2026a)	✓		98.9	98.1	96.0	**100**	85.3	91.0
Qwen3.5-0.8B (RL)	✓		98.6	96.6	94.8	**100**	83.7	88.8
Qwen3.5-2B (RL)	✓		98.4	97.0	94.1	**100**	84.5	89.2
Qwen3.5-4B (RL)	✓		**99.1**	98.0	95.9	**100**	86.0	91.2
Reflect-VLA-0.8B	✓		98.8	96.7	95.8	**100**	83.2	89.3
Reflect-VLA-2B	✓		98.9	97.0	96.3	**100**	84.2	90.0
Reflect-VLA-4B	✓		98.9	98.0	96.7	**100**	86.0	**91.5**

Table 1: Comparison with state-of-the-art methods on NAVSIMv1 with PDMS.

Method	NC ↑	DAC ↑	DDC ↑	TLC ↑	EP ↑	TTC ↑	LK ↑	HC ↑	EC ↑	EPDMS ↑
Ego Status	93.1	77.9	92.7	99.6	86.0	91.5	89.4	98.3	85.4	64.0
TransFuser (Chitta et al. 2022)	96.9	89.9	97.8	99.7	87.1	95.4	92.7	98.3	87.2	76.7
DiffusionDrive (Liao et al. 2025)	98.2	95.9	99.4	**99.8**	87.5	97.3	96.8	98.3	**87.7**	84.5
HydraMDP++ (Li et al. 2024)	98.5	98.5	99.5	99.7	87.4	97.9	95.8	98.2	75.7	85.6
DriveVLA-W0-7B (Li et al. 2025a)	98.5	**99.1**	98.0	99.7	86.4	98.1	93.2	97.9	58.9	86.1
Reflect-VLA-0.8B	98.8	96.7	99.1	99.7	87.5	98.3	96.5	98.3	86.2	88.8
Reflect-VLA-2B	**98.9**	97.0	99.3	99.7	88.1	98.4	96.5	**98.4**	86.8	89.5
Reflect-VLA-4B	**98.9**	97.9	**99.6**	**99.8**	**88.8**	**98.6**	**97.9**	98.3	87.5	**91.2**

Table 2: Comparison with state-of-the-art methods on NAVSIMv2 with EPDMS.

4 Experiment

4.1 Implementation details

Dataset. We evaluate trajectory generation on real-world driving scenarios using NAVSIM v1 and NAVSIM v2 (Dauner et al. 2024; Cao et al. 2025), both of which provide simulation-based planning metrics. NAVSIM v2 further extends the scope of evaluation to traffic-rule compliance, lane-level behavior, and temporal consistency. To assess whether the learned driving capabilities generalize to interactive decision making, we additionally evaluate Reflect-VLA on the closed-loop Bench2Drive benchmark (Jia et al. 2024). Together, these benchmarks provide a comprehensive evaluation of both trajectory-level planning quality and closed-loop driving performance. Detailed dataset splits and evaluation settings are provided in Appendix D.

Metrics. To evaluate trajectory prediction and closed-loop driving performance, we employ benchmark-specific core metrics. We use the Predictive Driver Model Score (PDMS) for NAVSIM v1 (Dauner et al. 2024) and the Extended Predictive Driver Model Score (EPDMS) for NAVSIM v2 (Cao et al. 2025). For Bench2Drive, we report Driving Score (DS), Success Rate (SR), Efficiency (Effi), and Comfort (Comf). Detailed metric definitions are provided in Appendix E.

Training Details. We use Qwen3.5-0.8B/2B/4B (Team 2026) as the base model. Training consists of three stages: (1) driving-knowledge pretraining for 2 epochs with batch size 1 and a learning rate of 1×10^{-5}; (2) fine-tuning on the planning, refinement, and rejection-sampling data for 2 epochs with batch size 4 and a learning rate of 4×10^{-5}; and (3) on-policy self-distillation for 2 epochs with batch size 2 and learning rates of 8×10^{-6} and 2×10^{-6}, respectively. Training is conducted on 8 NVIDIA H20 GPUs. Further details are provided in Appendix E.

4.2 Performance Comparison

NAVSIM Benchmark. Table 1 and Table 2 report the planning performance on NAVSIM v1 and NAVSIM v2, respectively. On NAVSIM v1, Reflect-VLA consistently improves

Method	DS↑	SR (%)↑	Effi↑	Comf↑
End-to-End Methods				
VAD (Jiang et al. 2023)	42.35	15.00	157.94	**46.01**
UniAD-Base (Hu et al. 2023)	45.81	16.36	129.21	43.58
DriveAdapter (Jia et al. 2023)	64.22	33.08	70.22	16.01
Vision-Language-Action Methods				
ReasonPlan (Liu et al. 2025)	64.01	34.55	180.64	25.63
MindDriver (Zhang et al. 2026)	65.48	39.55	143.21	34.63
ORION (Fu et al. 2025)	77.74	54.62	151.48	17.38
AutoVLA (Zhou et al. 2025b)	78.84	57.73	146.93	39.33
Reflect-VLA-0.8B	76.03	54.54	183.47	28.06
Reflect-VLA-2B	77.39	55.90	174.15	27.89
Reflect-VLA-4B	**80.15**	**59.09**	**212.20**	21.92

Table 3: Comparison with state-of-the-art methods on the Bench2Drive benchmark with DS (Driving Score), SR (Success Rate), Effi (Efficiency), and Comf (Comfortness).

Method	Student	Teacher	OPSD
Reflect-VLA-0.8B	86.3	92.4	89.3
Reflect-VLA-2B	87.4	92.7	90.0
Reflect-VLA-4B	88.4	93.2	91.5

Table 4: Planning performance of the student, teacher, and student after Dual-OPSD on NAVSIMv1 using PDMS, where student and teacher refer to the same model after three-stage SFT performing trajectory prediction without and with guidance, respectively.

over the corresponding RL baselines across all model scales. Reflect-VLA-0.8B, 2B, and 4B achieve PDMS scores of 89.3, 90.0, and 91.5, outperforming their corresponding RL baselines by 0.5, 0.8, and 0.3 points, respectively. Notably, Reflect-VLA-4B achieves the best overall PDMS score, with improved TTC while maintaining strong performance on other metrics. These results validate the effectiveness and scalability of progressive guidance internalization: rather than relying solely on sparse scalar rewards, Reflect-VLA converts structured reflective guidance into fine-grained distributional supervision, enabling the policy to directly absorb trajectory-correction preferences. Together with its reduced training cost discussed below, Reflect-VLA provides a favorable balance between planning performance and optimization efficiency.

Bench2Drive Benchmark. Table 3 shows the closed-loop driving performance on the Bench2Drive benchmark. Our Reflect-VLA-4B achieves the best Driving Score (DS) of 80.15, Efficiency (Effi) of 212.20, and Success Rate (SR) of 59.09%, outperforming existing baselines, including AutoVLA. This indicates that the guidance internalization possesses strong generalization and effectiveness in closed-loop evaluations involving complex interactions.

Effectiveness of Reflective Guidance. Table 4 compares the performance of the student, the guidance-conditioned teacher, and the student after Dual-OPSD. Across all model scales, reflective guidance consistently improves planning performance, with the teacher outperforming the unguided student by 6.1, 5.3, and 4.8 PDMS for the 0.8B, 2B, and 4B models, respectively. After Dual-OPSD, the student gains 3.0, 2.6, and 3.1 PDMS over its original performance, substantially narrowing the gap to the teacher. These results demonstrate that Dual-OPSD effectively internalizes the planning knowledge elicited by reflective guidance, enabling Reflect-VLA to retain most of its benefits while requiring no guidance during inference.

Base Model	SFT	GRPO	OPSD
Qwen3.5-0.8B	0.11	5.3	1.3
Qwen3.5-2B	0.26	22.3	1.7
Qwen3.5-4B	0.43	33.4	2.4

Table 5: Comparison of training time (hours) per 10k samples for SFT, GRPO, and OPSD across different model scales.

Training Efficiency. Table 5 compares the training time of SFT, GRPO, and OPSD across different model scales. It shows that OPSD consistently requires less training time than GRPO. For the 0.8B model, OPSD reduces the training time per 10,000 samples from 5.3 to 1.3 hours, corresponding to a $4.1\times$ speedup. This advantage increases with model scale: OPSD is $13.1\times$ faster for the 2B model and $13.9\times$ faster for the 4B model, requiring only 1.7 and 2.4 hours, respectively. Compared with GRPO, which requires expensive online exploration and iterative reward-based optimization, OPSD transfers guidance-conditioned knowledge through dense token-level supervision, enabling substantially more efficient training while maintaining competitive planning performance.

4.3 Qualitative Analysis

Figure 3 presents qualitative comparisons among the unguided student, the guidance-conditioned teacher, and the distilled student across representative driving scenarios. In the first scenario, the unguided student predicts a trajectory with potential collision risk, whereas the teacher produces a safer trajectory with reduced longitudinal progress. Dual-OPSD internalizes this corrective behavior into the student, enabling safer trajectory generation without reflective guidance at inference time. The token-level distributions provide further insight into this transfer. Given the same student-generated trajectory prefix, the teacher assigns the highest probability to the more conservative special trajectory token <|x_208|>, while the student initially favors <|x_215|>. Since the trajectory tokens encode relative displacements quantized at a resolution of 0.01 m, these tokens correspond to forward displacements of 2.08 m and 2.15 m, respectively. By minimizing the reverse KL divergence, Dual-OPSD aligns the student distribution with the teacher distribution and transfers this fine-grained correction preference. The second scenario shows a similar effect for a drivable-area violation: the teacher corrects the lateral deviation, and the distilled student successfully reproduces a feasible trajectory without guidance. Additional qualitative results are provided in Appendix F.

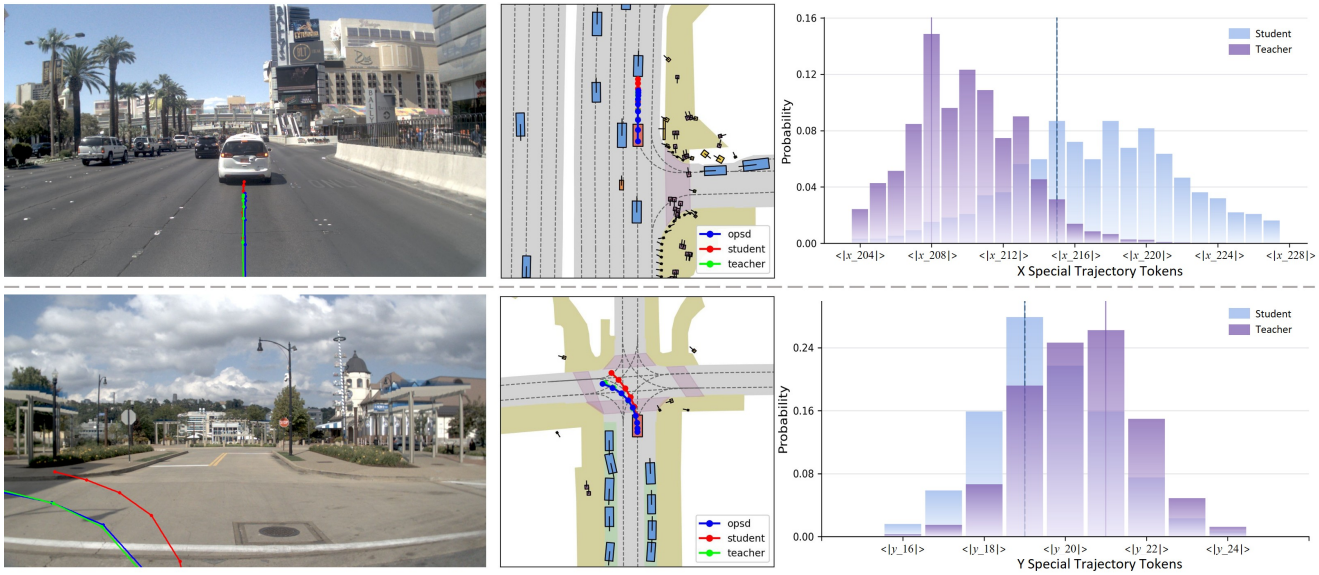


Figure 3: **Qualitative comparisons and token-level analysis.** Left and middle: trajectories generated by the unguided student, guidance-conditioned teacher, and distilled student. Right: distributions over special trajectory tokens at critical decoding steps under the same student-generated prefix. Each token encodes a relative displacement quantized at 0.01 m resolution; e.g., <|x_208|> represents a 2.08 m forward displacement for the next waypoint.

Model	NC↑	DAC↑	TTC↑	CF↑	EP↑	PDMS↑
SFT	98.5	96.3	94.9	100	81.7	88.1
Pre+SFT	98.7	96.5	95.4	100	82.1	88.5
Pre+SFT+RS	98.6	96.4	95.3	100	82.2	88.4
Pre+SFT+OPSD	98.9	97.1	95.8	100	84.7	90.2
Pre+SFT+RS+OPSD	**98.9**	**98.0**	**96.7**	**100**	**86.0**	**91.5**

Table 6: Ablation study of training stages. We evaluate the effect of pretraining, supervised fine-tuning, rejection sampling and on-policy self-distillation on driving performance using NAVSIM evaluation metrics with the 4B model.

ID	Trajectory Verdict	Planning Diagnosis	Expert Alignment	Refinement Directive	PDMS↑
1		✓	✓	✓	93.1
2	✓		✓	✓	92.4
3	✓	✓		✓	92.1
4	✓	✓	✓		92.9
5	✓	✓	✓	✓	**93.2**

Table 7: Ablation study of reflective guidance components for the teacher model. We report the NAVSIM PDMS of the 4B teacher when conditioned on different combinations of guidance components.

4.4 Ablation Studies

Ablation on Training Stages. Table 6 examines the contribution of each training stage. Pretraining improves PDMS from 88.1 to 88.5, confirming the benefit of domain-specific initialization. Self-Refinement Rejection Sampling alone slightly reduces PDMS to 88.4, likely due to partial forgetting of base planning ability during further training on refined samples. However, combining it with OPSD raises the score from 90.2 to 91.5, indicating that refinement samples derived from the model's own rollouts better align the guidance-conditioned behavior with the failure distribution encountered during on-policy distillation, thereby providing more effective supervision for guidance internalization.

Ablation on Reflective Guidance Components. Table 7 evaluates the contribution of each reflective guidance component. The complete design achieves the best PDMS of 93.2, while removing any component degrades performance, confirming the effectiveness of our structured guidance design. In particular, *Expert Alignment* and *Planning Diagnosis* yield the largest gains, highlighting the importance of identifying planning failures and relating them to deviations from expert behavior.

5 Conclusion

In this paper, we introduce Reflect-VLA, a training framework that internalizes reflective guidance into VLA models for autonomous driving. Through dual-path on-policy self-distillation, Reflect-VLA transfers token-level corrective distributions from guidance-conditioned generation to direct trajectory prediction, enabling strong planning without explicit guidance or iterative refinement at inference time. Experiments on NAVSIM and Bench2Drive demonstrate competitive driving performance and up to a $10\times$ training speedup over reinforcement-learning baselines, while retaining efficient, guidance-free single-pass planning across different model scales.

References

Agarwal, R.; Vieillard, N.; Zhou, Y.; Stanczyk, P.; Ramos Garea, S.; Geist, M.; and Bachem, O. 2024. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, volume 2024, 21246–21263.

Cao, W.; Hallgarten, M.; Li, T.; Dauner, D.; Gu, X.; Wang, C.; Miron, Y.; Aiello, M.; Li, H.; Gilitschenski, I.; et al. 2025. Pseudo-simulation for autonomous driving. *arXiv preprint arXiv:2506.04218*.

Chen, L.; Wu, P.; Chitta, K.; Jaeger, B.; Geiger, A.; and Li, H. 2024. End-to-end autonomous driving: Challenges and frontiers. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.

Chitta, K.; Prakash, A.; Jaeger, B.; Yu, Z.; Renz, K.; and Geiger, A. 2022. Transfuser: Imitation with transformer-based sensor fusion for autonomous driving. *IEEE transactions on pattern analysis and machine intelligence*, 45(11): 12878–12895.

Dauner, D.; Hallgarten, M.; Li, T.; Weng, X.; Huang, Z.; Yang, Z.; Li, H.; Gilitschenski, I.; Ivanovic, B.; Pavone, M.; et al. 2024. Navsim: Data-driven non-reactive autonomous vehicle simulation and benchmarking. *Advances in Neural Information Processing Systems*, 37: 28706–28719.

Ding, X.; Han, J.; Xu, H.; Liang, X.; Zhang, W.; and Li, X. 2024. Holistic autonomous driving understanding by bird's-eye-view injected multi-modal large models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 13668–13677.

Fu, H.; Zhang, D.; Zhao, Z.; Cui, J.; Liang, D.; Zhang, C.; Zhang, D.; Xie, H.; Wang, B.; and Bai, X. 2025. Orion: A holistic end-to-end autonomous driving framework by vision-language instructed action generation. *arXiv preprint arXiv:2503.19755*.

Hu, Y.; Yang, J.; Chen, L.; Li, K.; Sima, C.; Zhu, X.; Chai, S.; Du, S.; Lin, T.; Wang, W.; et al. 2023. Planning-oriented autonomous driving. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 17853–17862.

Huang, Z.; Tang, T.; Chen, S.; Lin, S.; Jie, Z.; Ma, L.; Wang, G.; and Liang, X. 2024. Making large language models better planners with reasoning-decision alignment. In *European Conference on Computer Vision*, 73–90. Springer.

Ishaq, A.; Lahoud, J.; More, K.; Thawakar, O.; Thawkar, R.; Dissanayake, D.; Ahsan, N.; Li, Y.; Khan, F. S.; Cholakkal, H.; et al. 2025. Drivelmm-o1: A step-by-step reasoning dataset and large multimodal model for driving scenario understanding. *arXiv preprint arXiv:2503.10621*.

Jia, X.; Gao, Y.; Chen, L.; Yan, J.; Liu, P. L.; and Li, H. 2023. Driveadapter: Breaking the coupling barrier of perception and planning in end-to-end autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 7953–7963.

Jia, X.; Yang, Z.; Li, Q.; Zhang, Z.; and Yan, J. 2024. Bench2drive: Towards multi-ability benchmarking of closed-loop end-to-end autonomous driving. *Advances in Neural Information Processing Systems*, 37: 819–844.

Jiang, B.; Chen, S.; Liao, B.; Zhang, X.; Yin, W.; Zhang, Q.; Huang, C.; Liu, W.; and Wang, X. 2024. Senna: Bridging large vision-language models and end-to-end autonomous driving. *arXiv preprint arXiv:2410.22313*.

Jiang, B.; Chen, S.; Xu, Q.; Liao, B.; Chen, J.; Zhou, H.; Zhang, Q.; Liu, W.; Huang, C.; and Wang, X. 2023. Vad: Vectorized scene representation for efficient autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 8340–8350.

Jiang, B.; Chen, S.; Zhang, Q.; Liu, W.; and Wang, X. 2025a. Alphadrive: Unleashing the power of vlms in autonomous driving via reinforcement learning and reasoning. *arXiv preprint arXiv:2503.07608*.

Jiang, S.; Huang, Z.; Qian, K.; Luo, Z.; Zhu, T.; Zhong, Y.; Tang, Y.; Kong, M.; Wang, Y.; Jiao, S.; et al. 2025b. A Survey on Vision-Language-Action Models for Autonomous Driving. *arXiv preprint arXiv:2506.24044*.

Li, Y.; Shang, S.; Liu, W.; Zhan, B.; Wang, H.; Wang, Y.; Chen, Y.; Wang, X.; An, Y.; Tang, C.; et al. 2025a. DriveVLA-W0: World Models Amplify Data Scaling Law in Autonomous Driving. *arXiv preprint arXiv:2510.12796*.

Li, Y.; Wang, Y.; Liu, Y.; He, J.; Fan, L.; and Zhang, Z. 2025b. End-to-end driving with online trajectory evaluation via bev world model. *arXiv preprint arXiv:2504.01941*.

Li, Y.; Xiong, K.; Guo, X.; Li, F.; Yan, S.; Xu, G.; Zhou, L.; Chen, L.; Sun, H.; Wang, B.; et al. 2025c. ReCogDrive: A Reinforced Cognitive Framework for End-to-End Autonomous Driving. *arXiv preprint arXiv:2506.08052*.

Li, Z.; Li, K.; Wang, S.; Lan, S.; Yu, Z.; Ji, Y.; Li, Z.; Zhu, Z.; Kautz, J.; Wu, Z.; et al. 2024. Hydra-mdp: End-to-end multimodal planning with multi-target hydra-distillation. *arXiv preprint arXiv:2406.06978*.

Liao, B.; Chen, S.; Yin, H.; Jiang, B.; Wang, C.; Yan, S.; Zhang, X.; Li, X.; Zhang, Y.; Zhang, Q.; et al. 2025. Diffusiondrive: Truncated diffusion model for end-to-end autonomous driving. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, 12037–12047.

Liu, X.; Zhong, Z.; Guo, Y.; Liu, Y.-F.; Su, Z.; Zhang, Q.; Wang, J.; Gao, Y.; Zheng, Y.; Lin, Q.; et al. 2025. ReasonPlan: Unified Scene Prediction and Decision Reasoning for Closed-loop Autonomous Driving. *arXiv preprint arXiv:2505.20024*.

Lu, J.; Guan, J.; Huang, Z.; Li, J.; Li, G.; Kong, L.; Li, Y.; Wang, H.; Xu, S.; Luo, Y.; et al. 2026. Xiaomi OneVL: One-Step Latent Reasoning and Planning with Vision-Language Explanation. *arXiv preprint arXiv:2604.18486*.

Luo, Y.; Li, F.; Chen, Q.; Xu, S.; Liu, J.; Song, Z.; Yang, Z.-x.; and Wen, F. 2026a. Unleashing vla potentials in autonomous driving via explicit learning from failures. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 24833–24842.

Luo, Y.; Li, F.; Xu, S.; Ji, Y.; Zhang, Z.; Wang, B.; Shen, Y.; Cui, J.; Chen, L.; Chen, G.; et al. 2026b. Last-vla: Thinking in latent spatio-temporal space for vision-language-action in autonomous driving. *arXiv preprint arXiv:2603.01928*.

Luo, Y.; Li, F.; Xu, S.; Lai, Z.; Yang, L.; Chen, Q.; Luo, Z.; Xie, Z.; Jiang, S.; Liu, J.; et al. 2025. AdaThinkDrive: Adaptive Thinking via Reinforcement Learning for Autonomous Driving. *arXiv preprint arXiv:2509.13769*.

Marcu, A.-M.; Chen, L.; Hünermann, J.; Karnsund, A.; Hanotte, B.; Chidananda, P.; Nair, S.; Badrinarayanan, V.; Kendall, A.; Shotton, J.; et al. 2024. LingoQA: Visual question answering for autonomous driving. In *European Conference on Computer Vision*, 252–269. Springer.

Qian, T.; Chen, J.; Zhuo, L.; Jiao, Y.; and Jiang, Y.-G. 2024. Nuscenes-qa: A multi-modal visual question answering benchmark for autonomous driving scenario. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 4542–4550.

Sang, H.; Xu, Y.; Zhou, Z.; He, R.; Wang, Z.; and Sun, J. 2026. CRISP: Compressed Reasoning via Iterative Self-Policy Distillation. *arXiv preprint arXiv:2603.05433*.

Sima, C.; Renz, K.; Chitta, K.; Chen, L.; Zhang, H.; Xie, C.; Beißwenger, J.; Luo, P.; Geiger, A.; and Li, H. 2024. Drivelm: Driving with graph visual question answering. In *European conference on computer vision*, 256–274. Springer.

Team, Q. 2026. Qwen3.5: Accelerating Productivity with Native Multimodal Agents.

Xing, S.; Qian, C.; Wang, Y.; Hua, H.; Tian, K.; Zhou, Y.; and Tu, Z. 2025. Openemma: Open-source multimodal model for end-to-end autonomous driving. In *Proceedings of the Winter Conference on Applications of Computer Vision*, 1001–1009.

Xu, Z.; Zhang, Y.; Xie, E.; Zhao, Z.; Guo, Y.; Wong, K.-Y. K.; Li, Z.; and Zhao, H. 2024. DriveGPT4: Interpretable End-to-end Autonomous Driving via Large Language Model. *arXiv preprint arXiv:2310.01412*.

Yang, L.; Huang, J.; Huang, Z.; Liu, S.; and Yang, H. 2026. Judge, Then Drive: A Critic-Centric Vision Language Action Framework for Autonomous Driving. *arXiv preprint arXiv:2604.27366*.

Ye, T.; Dong, L.; Wu, X.; Huang, S.; and Wei, F. 2026. On-Policy Context Distillation for Language Models. *arXiv preprint arXiv:2602.12275*.

Yuan, Z.; Qian, C.; Tang, J.; Chen, R.; Song, Z.; Sun, L.; Chu, X.; Cai, Y.; Zhang, D.; and Li, S. 2025. AutoDrive-R^2: Incentivizing Reasoning and Self-Reflection Capacity for VLA Model in Autonomous Driving. *arXiv preprint arXiv:2509.01944*.

Zeng, S.; Chang, X.; Xie, M.; Liu, X.; Bai, Y.; Pan, Z.; Xu, M.; Wei, X.; and Guo, N. 2025. Futuresightdrive: Thinking visually with spatio-temporal cot for autonomous driving. *arXiv preprint arXiv:2505.17685*.

Zhang, L.; Yuan, Y.; Wu, C.; Chang, X.; Cai, X.; Zeng, S.; Shi, L.; Wang, S.; Zhang, H.; and Xu, M. 2026. MindDriver: Introducing Progressive Multimodal Reasoning for Autonomous Driving. *arXiv preprint arXiv:2602.21952*.

Zhao, R.; Yuan, Q.; Li, J.; Hu, H.; Li, Y.; Zheng, C.; and Gao, F. 2025. Sce2drivex: A generalized mllm framework for scene-to-drive learning. *arXiv preprint arXiv:2502.14917*.

Zhao, S.; Xie, Z.; Liu, M.; Huang, J.; Pang, G.; Chen, F.; and Grover, A. 2026. Self-Distilled Reasoner: On-Policy Self-Distillation for Large Language Models. *arXiv preprint arXiv:2601.18734*.

Zheng, W.; Xia, Z.; Huang, Y.; Zuo, S.; Zhou, J.; and Lu, J. 2024. Doe-1: Closed-loop autonomous driving with large world model. *arXiv preprint arXiv:2412.09627*.

Zhong, Z.; Yan, H.; Li, J.; He, J.; Zhang, T.; and Li, H. 2026. VLA-OPD: Bridging Offline SFT and Online RL for Vision-Language-Action Models via On-Policy Distillation. *arXiv preprint arXiv:2603.26666*.

Zhou, X.; Han, X.; Yang, F.; Ma, Y.; Tresp, V.; and Knoll, A. 2025a. Opendrivevla: Towards end-to-end autonomous driving with large vision language action model. *arXiv preprint arXiv:2503.23463*.

Zhou, Z.; Cai, T.; Zhao, S. Z.; Zhang, Y.; Huang, Z.; Zhou, B.; and Ma, J. 2025b. AutoVLA: A Vision-Language-Action Model for End-to-End Autonomous Driving with Adaptive Reasoning and Reinforcement Fine-Tuning. *arXiv preprint arXiv:2506.13757*.

Zuo, S.; Xie, Z.; Zheng, W.; Xu, S.; Li, F.; Li, H.; Chen, L.; Yang, Z.-X.; and Lu, J. 2026. Dvgt-2: Vision-geometry-action model for autonomous driving at scale. *arXiv preprint arXiv:2604.00813*.

Reproducibility Checklist

Instructions for Authors:
This document outlines key aspects for assessing reproducibility. Please provide your input by editing this `.tex` file directly.

For each question (that applies), replace the "Type your response here" text with your answer.

Example: If a question appears as

```
\question{Proofs of all novel
claims are included}
{(yes/partial/no)}
Type your response here
```

you would change it to:

```
\question{Proofs of all novel
claims are included}
{(yes/partial/no)}
yes
```

Please make sure to:

- Replace ONLY the "Type your response here" text and nothing else.
- Use one of the options listed for that question (e.g., **yes**, **no**, **partial**, or **NA**).
- **Not** modify any other part of the `\question` command or any other lines in this document.

You can `\input` this .tex file right before `\end{document}` of your main file or compile it as a stand-alone document. Check the instructions on your conference's website to see if you will be asked to provide this checklist with your paper or separately.

1. General Paper Structure

1.1. Includes a conceptual outline and/or pseudocode description of AI methods introduced (yes/partial/no/NA) partial

1.2. Clearly delineates statements that are opinions, hypothesis, and speculation from objective facts and results (yes/no) yes

1.3. Provides well-marked pedagogical references for less-familiar readers to gain background necessary to replicate the paper (yes/no) yes

2. Theoretical Contributions

2.1. Does this paper make theoretical contributions? (yes/no) no

If yes, please address the following points:

2.2. All assumptions and restrictions are stated clearly and formally (yes/partial/no) no

2.3. All novel claims are stated formally (e.g., in theorem statements) (yes/partial/no) no

2.4. Proofs of all novel claims are included (yes/partial/no) no

2.5. Proof sketches or intuitions are given for complex and/or novel results (yes/partial/no) no

2.6. Appropriate citations to theoretical tools used are given (yes/partial/no) no

2.7. All theoretical claims are demonstrated empirically to hold (yes/partial/no/NA) NA

2.8. All experimental code used to eliminate or disprove claims is included (yes/no/NA) NA

3. Dataset Usage

3.1. Does this paper rely on one or more datasets? (yes/no) yes

If yes, please address the following points:

3.2. A motivation is given for why the experiments are conducted on the selected datasets (yes/partial/no/NA) yes

3.3. All novel datasets introduced in this paper are included in a data appendix (yes/partial/no/NA) NA

3.4. All novel datasets introduced in this paper will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no/NA) NA

3.5. All datasets drawn from the existing literature (potentially including authors' own previously published work) are accompanied by appropriate citations (yes/no/NA) yes

3.6. All datasets drawn from the existing literature (potentially including authors' own previously published work) are publicly available (yes/partial/no/NA) yes

3.7. All datasets that are not publicly available are described in detail, with explanation why publicly available alternatives are not scientifically satisficing (yes/partial/no/NA) NA

4. Computational Experiments

4.1. Does this paper include computational experiments? (yes/no) yes

If yes, please address the following points:

4.2. This paper states the number and range of values tried per (hyper-) parameter during development of the paper, along with the criterion used for selecting the final parameter setting (yes/partial/no/NA) partial

4.3. Any code required for pre-processing data is included

in the appendix (yes/partial/no) no

4.4. All source code required for conducting and analyzing the experiments is included in a code appendix (yes/partial/no) no

4.5. All source code required for conducting and analyzing the experiments will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no) partial

4.6. All source code implementing new methods have comments detailing the implementation, with references to the paper where each step comes from (yes/partial/no) partial

4.7. If an algorithm depends on randomness, then the method used for setting seeds is described in a way sufficient to allow replication of results (yes/partial/no/NA) partial

4.8. This paper specifies the computing infrastructure used for running experiments (hardware and software), including GPU/CPU models; amount of memory; operating system; names and versions of relevant software libraries and frameworks (yes/partial/no) partial

4.9. This paper formally describes evaluation metrics used and explains the motivation for choosing these metrics (yes/partial/no) partial

4.10. This paper states the number of algorithm runs used to compute each reported result (yes/no) yes

4.11. Analysis of experiments goes beyond single-dimensional summaries of performance (e.g., average; median) to include measures of variation, confidence, or other distributional information (yes/no) yes

4.12. The significance of any improvement or decrease in performance is judged using appropriate statistical tests (e.g., Wilcoxon signed-rank) (yes/partial/no) yes

4.13. This paper lists all final (hyper-)parameters used for each model/algorithm in the paper's experiments (yes/partial/no/NA) partial

A Appendix

In this appendix, we provide additional experimental results, data construction details, implementation settings, and qualitative analyses to complement the main paper. **Section B** addresses several potential questions about the proposed framework. **Section C** reports the open-loop planning results on nuScenes and ablations on the top-k distillation support, KL-divergence direction, and trajectory-routing threshold. **Section D** details trajectory tokenization, Reflective Guidance construction, pretraining data, and the construction of planning, refinement, and self-refinement rejection-sampling data. **Section E** describes the model architecture, training parameters, hardware configuration, and benchmark-specific evaluation metrics. Finally, **Section F** presents detailed qualitative examples and token-level analyses of how Dual-OPSD transfers corrective planning preferences from the guidance-conditioned teacher to the student.

B Question

Q1: Why not directly use Reflective Guidance during inference?

A: Constructing Reflective Guidance requires an initial trajectory and its evaluation results, which introduces an additional prediction–evaluation– refinement loop. Reflect-VLA instead treats the guidance as training-time privileged information and internalizes its corrective preferences, enabling single-pass inference without an online evaluator.

Q2: Why does Dual-OPSD use separate learning paths for reliable and risky trajectories?

A: Teacher correction is useful for risky trajectories but may unnecessarily modify predictions that are already reliable. Dual-OPSD therefore distills guidance-conditioned teacher distributions for risky rollouts while applying self-preservation to reliable ones, balancing correction and behavior retention.

Q3: Why is Self-Refinement Rejection Sampling needed before Dual-OPSD?

A: Offline refinement data may not cover the failure patterns produced by the model itself. Self-Refinement Rejection Sampling exposes the teacher to model-generated failures and retains only evaluator-verified improvements, making its corrective behavior better aligned with the student rollout distribution.

Q4: Does Reflective Guidance require an additional VLM or manual feedback?

A: No. Reflective Guidance is constructed through deterministic templates using trajectory-evaluation results, geometric differences from a reference trajectory, and the navigation command. It therefore provides structured corrective supervision without introducing another VLM or requiring manual annotation.

C Additional Results

1 Main Results

nuScenes Benchmark. Table 8 reports the open-loop planning results on nuScenes. Reflect-VLA-4B achieves state-of-the-art performance among VLA-based methods, with the lowest average L2 error of 0.41 m and collision rate of 0.14%. These results indicate that our approach improves geometric trajectory accuracy while maintaining strong planning safety. The smaller Reflect-VLA-2B model also achieves competitive results, with an average L2 error of 0.52 m and collision rate of 0.20%, outperforming OpenDriveVLA and approaching FSDrive. Even the 0.8B variant remains comparable to AutoVLA, demonstrating consistent effectiveness across different model scales.

2 Ablation Studies

Effect of Top-k Distillation. Table 9 evaluates the size of the teacher-distribution support retained during token-level distillation. Increasing k from 16 to 64 improves PDMS from 90.2 to 91.5, since the larger support preserves more informative teacher preferences. Further increasing k to 256 or the full vocabulary yields only marginal gains, indicating diminishing returns once the dominant teacher distribution is sufficiently covered. We therefore use $k = 64$ as the distillation support size.

Effect of KL-Divergence Direction. Table 10 shows that reverse KL improves PDMS from 91.0 to 91.5. Reverse KL encourages the student to concentrate on the teacher's high-probability corrective tokens and suppress unlikely alternatives. In contrast, forward KL favors broader distribution coverage, which may dilute the teacher's dominant correction preferences. We therefore adopt reverse KL for token-level distillation.

Effect of the Routing Threshold. Table 11 shows that increasing s from 0.65 to 0.85 gradually improves PDMS, as more suboptimal trajectories are routed to distillation and aligned with the teacher's corrective preferences. However, a larger threshold of 0.95 also distills many already reliable trajectories, where unnecessary teacher intervention may disrupt effective student behaviors. We therefore set $s = 0.85$ to balance correction and preservation.

D Data Preparation Details

1 Trajectory Tokenization

The future trajectory is represented as $o = \{(x_t, y_t, h_t)\}_{t=1}^{T}$, where x_t, y_t, and h_t denote the longitudinal position, lateral position, and heading, respectively. To support autoregressive trajectory generation, we convert the continuous trajectory into structured tokens. Specifically, we prepend the current ego pose $(x_0, y_0, h_0) = (0, 0, 0)$ and represent the motion between consecutive timesteps by its relative displacement:

$$(\Delta x_t, \Delta y_t, \Delta h_t) = (x_t - x_{t-1},\ y_t - y_{t-1},\ h_t - h_{t-1}), \quad (11)$$

Each displacement component $d \in \{x, y, h\}$ is clipped to a predefined range and uniformly quantized with resolution $\delta = 0.01$, corresponding to 0.01 m for the longitudinal and lateral displacements and 0.01 rad for the heading change:

$$k_t^d = \mathrm{round}\left(\frac{\mathrm{clip}(\Delta d_t, l_d, u_d)}{s}\right), \quad (12)$$

Method	VLM Model	L2 (m) ↓				Collision (%) ↓			
		1s	2s	3s	Avg.	1s	2s	3s	Avg.
Traditional End-to-end Methods									
UniAD* (Hu et al. 2023)	-	0.20	0.42	0.75	0.46	0.02	0.25	0.84	0.37
Vision-Language-Action Methods									
Doe-1 (Zheng et al. 2024)	Lumina-mGPT-7B	0.50	1.18	2.11	1.26	0.04	0.37	1.19	0.53
AutoVLA* (Zhou et al. 2025b)	Qwen2.5-VL-3B	0.28	0.66	1.16	0.70	0.14	0.25	0.53	0.31
RDA-Driver* (Huang et al. 2024)	LLaVA-7B	0.23	0.73	1.54	0.80	**0.00**	0.13	0.83	0.32
OpenDriveVLA* (Zhou et al. 2025a)	Qwen2.5VL-3B	0.19	0.58	1.24	0.67	0.02	0.18	0.70	0.30
FSDrive* (Zeng et al. 2025)	Qwen2-VL-2B	0.18	0.39	0.77	0.45	**0.00**	0.06	0.42	0.16
Reflect-VLA* (Ours)	Qwen3.5-0.8B	0.30	0.69	1.20	0.73	0.15	0.27	0.57	0.33
Reflect-VLA* (Ours)	Qwen3.5-2B	0.19	0.46	0.90	0.52	0.01	0.11	0.48	0.20
Reflect-VLA* (Ours)	Qwen3.5-4B	**0.16**	**0.35**	**0.71**	**0.41**	**0.00**	**0.05**	**0.37**	**0.14**

Table 8: Comparison with state-of-the-art methods on nuScenes. * indicates that ego status is used.

Top-K	NC↑	DAC↑	TTC↑	CF↑	EP↑	PDMS↑
Top-16	98.7	97.5	95.1	**100**	85.4	90.2
Top-64 (ours)	**98.9**	**98.0**	**96.7**	**100**	**86.0**	**91.5**
Top-256	**98.9**	97.9	96.5	**100**	**86.0**	91.4
Full Vocab	**98.9**	97.9	**96.7**	**100**	**86.0**	**91.5**

Table 9: Ablation study of the top-k truncation used for token-level distillation on NAVSIM with the 4B model.

Direction	NC↑	DAC↑	TTC↑	CF↑	EP↑	PDMS↑
Forward KL	**98.9**	97.8	95.9	**100**	85.8	91.0
Reverse KL(ours)	**98.9**	**98.0**	**96.7**	**100**	**86.0**	**91.5**

Table 10: Ablation study of the KL-divergence direction used for token-level distillation on NAVSIM with the 4B model.

We use $[l_x, u_x] = [-1.5, 15.0]$, $[l_y, u_y] = [-5.0, 5.0]$, and $[l_h, u_h] = [-0.5, 0.5]$, which cover the relative-motion ranges observed in the datasets. This gives $k_t^x \in [-150, 1500]$, $k_t^y \in [-500, 500]$, and $k_t^h \in [-50, 50]$, corresponding to 1,651, 1,001, and 101 discrete values for the longitudinal, lateral, and heading components, respectively.

Each quantized index is mapped to a dedicated special token. Non-negative indices are represented as <|x_k|>, <|y_k|>, and <|$theta_k$|>, while a negative index $-k$ uses an m prefix, such as <|x_mk|>. At timestep t, the three tokens are concatenated in longitudinal–lateral–heading order: $a_t = \tau_x(k_t^x)\tau_y(k_t^y)\tau_\theta(k_t^h)$. The complete trajectory is represented by the token sequence $[a_1, \ldots, a_T]$. For example, <|x_208|><|y_m4|><|$theta_1$|> represents the relative motion $(2.08, -0.04, 0.01)$ between two consecutive timesteps.

During inference, the generated indices are converted back into relative displacements and accumulated from the current ego pose:

$$(\hat{x}_t, \hat{y}_t, \hat{h}_t) = \sum_{i=1}^{t} s\,(k_i^x, k_i^y, k_i^h), \qquad t = 1, \ldots, T. \quad (13)$$

Threshold s	NC↑	DAC↑	TTC↑	CF↑	EP↑	PDMS↑
0.65	**98.9**	97.8	96.0	**100**	85.9	91.1
0.75	**98.9**	97.8	96.4	**100**	**86.0**	91.3
0.85 (ours)	**98.9**	**98.0**	**96.7**	**100**	**86.0**	**91.5**
0.95	**98.9**	97.8	96.2	**100**	**86.0**	91.2

Table 11: Ablation study of the PDMS routing threshold s in Dual-OPSD on NAVSIM with the 4B model.

This allows the autoregressively generated token sequence to be directly decoded into the predicted future trajectory.

2 Reflective Guidance Construction

We construct Reflective Guidance for low-quality trajectories using a deterministic template-based procedure. It relies on training-time privileged information, including trajectory-evaluation results, the navigation command, and a reference trajectory o^{ref}, without requiring an additional vision-language model. Given an initial trajectory o^{init}, we decode it into continuous waypoints when necessary and generate four components: **Trajectory Verdict**, **Planning Diagnosis**, **Expert Alignment**, and **Refinement Directive**.

Trajectory Verdict. The Trajectory Verdict converts evaluator outputs into an explicit reliability judgment. It identifies whether the initial prediction is problematic and summarizes the available PDMS, NC, DAC, TTC, EP, and CF metrics, providing quantitative context for subsequent correction.

Planning Diagnosis. The Planning Diagnosis identifies the dominant causes of failure, including collision risk, drivable-area violation, insufficient TTC margin, inadequate progress, and trajectory misalignment. Safety-critical failures are prioritized over efficiency-related errors, and only the most relevant diagnoses are retained. When event-level information is available, the first collision or boundary violation is localized to the corresponding waypoint and position.

Expert Alignment. The Expert Alignment compares o^{init} with o^{ref} and translates their geometric differences into qualitative adjustments. These adjustments indicate whether to preserve or shift the lateral position, increase or reduce longitudinal progress, and maintain or correct the final heading. The navigation command is included to keep the correction consistent with the intended maneuver.

Refinement Directive. The Refinement Directive converts the diagnosis and alignment results into an ordered correction plan. Collision and TTC failures first require creating a safer gap, while drivable-area violations require moving the path back into the valid lane corridor. The directive then specifies the required lateral, longitudinal, and heading adjustments, while preserving smooth intermediate waypoints and navigation intent. A primary focus, such as scene interpretation, lateral control, or longitudinal control, is assigned based on the dominant failure.

Construction Flow. We first normalize the trajectories and evaluator outputs and determine whether the initial trajectory requires correction. For a problematic trajectory, we extract the dominant failures, derive reference-relative adjustments, and instantiate the four components in the above order. The resulting guidance g is combined with the base input and initial trajectory as

$$q^{\text{gui}} = \left(q^{\text{base}}, o^{\text{init}}, g\right). \tag{14}$$

The expert or evaluator-verified trajectory is used as the refinement target, transforming scalar evaluation results and geometric discrepancies into structured, trajectory-specific corrective supervision.

3 Details of Pretraining Data

We collect several open-source driving QA datasets for driving-knowledge pretraining, including LingoQA, DriveGPT4, Senna, DriveLM, NuScenes-QA, NuInstruct, and DriveLMM.

LingoQA. LingoQA contains approximately 419.9K QA pairs from short driving video clips, covering scene description, localization, and reasoning.

DriveGPT4. DriveGPT4 provides visual instruction data from multi-frame driving videos, covering action interpretation, driving reasoning, question answering, and low-level control prediction.

Senna. Senna contains planning-oriented QA data over multi-view driving scenes, covering scene understanding, motion prediction, and high-level planning decisions.

DriveLM. DriveLM contains approximately 0.3M image-question pairs from 696 nuScenes scenes for graph-based driving-scene reasoning.

NuScenes-QA. NuScenes-QA provides approximately 460K QA pairs over multi-frame camera and LiDAR observations.

NuInstruct. NuInstruct contains approximately 91K multi-view video-QA pairs across 17 spatial, temporal, and multi-view reasoning tasks.

DriveLMM. DriveLMM provides over 18K training VQA examples with step-by-step reasoning for perception, prediction, and planning.

4 Details of SFT Data

Planning Data. For each dataset, we construct a base planning set in which each sample contains a front-view camera image, the ego-vehicle state including velocity and acceleration, a high-level navigation command (*Go Straight*, *Turn Left*, or *Turn Right*), a historical ego trajectory, and the ground-truth future trajectory. Both historical and future trajectories are sampled at 0.5-second intervals, and the future trajectory is converted into discrete special tokens for autoregressive prediction.

Guidance-Conditioned Refinement Data. Each refinement sample pairs the base driving input with a low-quality initial trajectory, its corresponding Reflective Guidance, and a reference trajectory. The initial trajectories are obtained from two complementary sources:

- **Planner-generated trajectories.** We collect trajectories produced by a classical planner and retain those exhibiting correctable failures in safety, drivable-area compliance, or progress.
- **Rule-perturbed trajectories.** We perturb valid trajectories along the longitudinal, lateral, or heading dimensions to construct representative planning failures while preserving a valid trajectory format.

All candidates are evaluated using the trajectory evaluator and map constraints, and low-quality but valid trajectories are selected as o^{init}. The ground-truth expert trajectory is used as the refinement target; alternatively, an evaluator-verified refined trajectory is retained when it resolves the failure and improves upon the initial trajectory.

Self-Refinement Rejection Sampling Data. We construct self-refinement data from the model's own rollouts. For each base input, the model first generates an unguided initial trajectory, which is evaluated to identify problematic cases. We then construct the corresponding Reflective Guidance and prompt the same model to generate a refined trajectory. A sample is retained only when the refined trajectory satisfies the trajectory and map constraints and achieves a higher evaluation score than the initial trajectory. When multiple valid refinements are available, we retain the highest-scoring one.

E Experimental Details

1 Training Parameters and Hardware Configuration.

We train Qwen3.5-0.8B/2B/4B on 8 NVIDIA H20 GPUs. Driving-knowledge pretraining is conducted for 2 epochs with a batch size of 1 and a learning rate of 1×10^{-5}. Planning-and-Refinement SFT is then performed on the base planning and guidance-conditioned refinement data, followed by Self-Refinement Rejection Sampling on the retained self-refinement data. These two stages use the same configuration of 2 epochs, a batch size of 4, and a learning rate of 4×10^{-5}. Dual-OPSD is subsequently performed for 2 epochs with a batch size of 2 and learning rates of 8×10^{-6} and 2×10^{-6} for the first and second epochs, respectively. During Dual-OPSD, on-policy rollouts are gener-

ated through greedy decoding, with the temperature set to 0. For token-level distillation, we retain the teacher's top-k support with $k = 64$ and minimize the reverse KL divergence over the truncated distribution. The routing threshold is set to $s = 0.85$. Reliable trajectories are optimized through self-preservation, whereas risky trajectories receive token-level supervision from the frozen guidance-conditioned teacher. Detailed hyperparameters are provided in Table 12.

Stage	Hyper-parameter	Value
Pretraining	Epochs	2
	Batch size	1
	Learning rate	1×10^{-5}
	Gradient accumulation steps	2
	Optimizer	Adam
	Weight decay	0.05
	Warmup ratio	0.1
SFT & Rejection Sampling	Epochs	2
	Batch size	4
	Learning rate	4×10^{-5}
	Gradient accumulation steps	2
	Optimizer	Adam
	Weight decay	0.05
	Warmup ratio	0.1
Dual-OPSD	Epochs	2
	Batch size	2
	Learning rate for epoch1	8×10^{-6}
	Learning rate for epoch2	2×10^{-6}
	Gradient accumulation steps	2
	Optimizer	Adam
	Weight decay	0.05
	Warmup ratio	0.1
	Routing threshold s	0.85
	Rollout temperature	0.0
	Distillation support top-k	64
	Distillation objective	Reverse KL
	Teacher update	Frozen
	Rollouts per input	1

Table 12: Training hyperparameters of Reflect-VLA.

2 Metrics.

We use benchmark-specific metrics to comprehensively evaluate planning performance. For the NAVSIM suite, we report the Predictive Driver Model Score (PDMS) on **NAVSIMv1** (Dauner et al. 2024) and the Extended Predictive Driver Model Score (EPDMS) on **NAVSIMv2** (Cao et al. 2025). For **Bench2Drive**, we evaluate interactive closed-loop performance using Driving Score (DS), Success Rate (SR), Efficiency (Effi), and Comfort (Comf). In addition, following UniAD (Hu et al. 2023), we evaluate open-loop trajectory quality on nuScenes using L2 error and collision rate.

For NAVSIMv1, PDMS combines five sub-metrics: No At-Fault Collision (NC), Drivable Area Compliance (DAC), Time-to-Collision (TTC), Comfort (CF), and Ego Progress (EP). It is computed as

$$PDMS = NC \times DAC \times \left(\frac{5 \times EP + 5 \times TTC + 2 \times CF}{12} \right), \tag{15}$$

where NC and DAC serve as multiplicative penalties, while EP, TTC, and CF contribute to the weighted planning score.

For NAVSIMv2, EPDMS extends the evaluation with additional safety, compliance, lane-level, and comfort metrics. It includes No At-Fault Collision (NC), Drivable Area Compliance (DAC), Driving Direction Compliance (DDC), Traffic Light Compliance (TLC), Ego Progress (EP), Time-to-Collision (TTC), Lane Keeping (LK), History Comfort (HC), and Extended Comfort (EC). The score is defined as

$$EPDMS = NC \times DAC \times DDC \times TLC \times \left(\frac{5 \times EP + 2 \times LK + 2 \times HC + 5 \times TTC + 2 \times EC}{16} \right), \tag{16}$$

For Bench2Drive, DS combines route completion with multiplicative penalties for driving infractions, while SR measures the proportion of routes completed within the time limit without traffic violations:

$$\begin{aligned} \text{Success Rate} &= \frac{n_{\text{success}}}{n_{\text{total}}}, \\ \text{Driving Score} &= \frac{1}{n_{\text{total}}} \sum_{i=1}^{n_{\text{total}}} \text{RC}_i \prod_{j=1}^{n_{i,\text{penalty}}} p_{i,j}. \end{aligned} \tag{17}$$

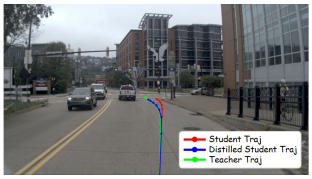
Here, n_{success} and n_{total} denote the numbers of successful and evaluated routes, respectively; RC_i is the route completion of route i, and $p_{i,j}$ is its j-th infraction penalty. Efficiency averages the ego-speed ratio relative to nearby vehicles, while Comfort reports the proportion of trajectory segments satisfying predefined kinematic smoothness thresholds.

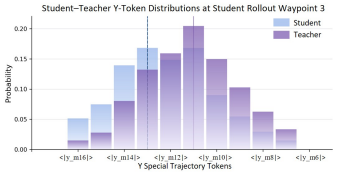
For nuScenes, we report the L2 error between the planned and expert trajectories together with the collision rate.

F Visualization Analysis

Figure 4 presents three representative failure cases. In each scenario, the unguided student produces a problematic trajectory, while the guidance-conditioned teacher generates a safer correction. After Dual-OPSD, the distilled student largely reproduces this behavior without requiring guidance at inference time. For the *Drivable-Area Violation*, the teacher shifts the trajectory away from the road boundary and back into the valid lane corridor. For the *Insufficient TTC Margin*, it reduces forward progress to maintain a safer distance from the conflict region. For the *At-Fault Collision Risk*, it increases clearance and postpones longitudinal progress until the hard safety constraint is resolved. The distilled student exhibits similar lateral and longitudinal adjustments in all three cases. The token-level distributions further reveal the transfer process. Under the same student-generated prefix, the teacher assigns greater probability to tokens corresponding to safer lateral or longitudinal motions, whereas the original student favors riskier alternatives. Dual-OPSD shifts the student toward these corrective preferences, resulting in safer token selection and improved trajectory generation

Scenario: Drivable-Area Violation





Query: <image> is the front view. Command: MOVE FORWARD. Velocity: [10.36, -0.09]. Acceleration: [-0.75, -0.53]. Historical trajectory: [-15.72, -0.53, 0.05], [-10.28, -0.26, 0.05], [-5.06, -0.05, 0.03].\n Plan the action tokens for the ego over the next four seconds, where each group of tokens corresponds to the vehicle's motion within a 0.5-second time window.

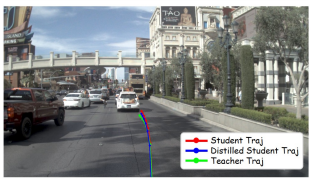
Guidance: 1. **Trajectory Verdict**: The initial trajectory is a problem trajectory that should be corrected before use. Metric summary: PDMS=0, NC=1, DAC=0, TTC=1, EP=0, CF=1. 2. **Planning Diagnosis**: **DAC=0** indicates the trajectory **leaves the legal drivable area**; the first boundary violation occurs near waypoint 6 around x=25.00m, y=-0.39m on the right side. The path must be pulled back inside the lane or connector boundary. The trajectory has a lateral bias relative to the expert-like path; the steering target should be corrected. 3. **Expert Alignment**: Compared with the expert trajectory, the initial path needs **shift the future waypoints moderately left**, increase longitudinal progress moderately over the planning horizon, and keep the final heading nearly unchanged. The navigation command is MOVE FORWARD. 4. **Refinement Directive**: First **restore drivable-area compliance by moving the path back inside the valid lane corridor**. Starting from the initial trajectory, **shift the future waypoints moderately left**, adjust longitudinal progress only after the hard safety constraint is fixed, and keep the final heading nearly unchanged. Keep intermediate waypoints smooth and preserve the navigation intent. Primary focus: Lateral Control.

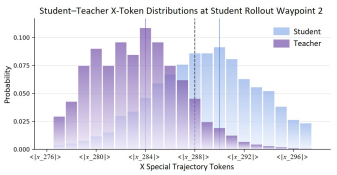
Student: <|x_489|><|y_m6|><|theta_m2|>, <|x_464|><|y_m11|><|theta_m1|>, <|x_434|><|y_m13|><|theta_0|>, <|x_403|><|y_m10|><|theta_1|>, <|x_371|><|y_m5|><|theta_2|>, <|x_339|><|y_6|><|theta_3|>, <|x_310|><|y_15|><|theta_3|>, <|x_278|><|y_22|><|theta_3|>

Teacher: <|x_489|><|y_m6|><|theta_m2|>, <|x_472|><|y_m10|><|theta_0|>, <|x_455|><|y_m6|><|theta_2|>, <|x_444|><|y_8|><|theta_4|>, <|x_428|><|y_26|><|theta_4|>, <|x_411|><|y_41|><|theta_3|>, <|x_392|><|y_49|><|theta_2|>, <|x_368|><|y_50|><|theta_1|>

Distilled Student: <|x_489|><|y_m6|><|theta_m2|>, <|x_457|><|y_m12|><|theta_0|>, <|x_419|><|y_m8|><|theta_2|>, <|x_386|><|y_6|><|theta_3|>, <|x_350|><|y_18|><|theta_4|>, <|x_321|><|y_30|><|theta_3|>, <|x_292|><|y_34|><|theta_2|>, <|x_259|><|y_32|><|theta_0|>

Scenario: Insufficient TTC Margin





Query: <image> is the front view. Command: MOVE FORWARD. Velocity: [5.99, -0.14]. Acceleration: [-0.01, 0.01]. Historical trajectory: [-9.00, -0.12, 0.05], [-5.96, -0.02, 0.02], [-2.97, 0.01, 0.00].\n Plan the action tokens for the ego over the next four seconds, where each group of tokens corresponds to the vehicle's motion within a 0.5-second time window.

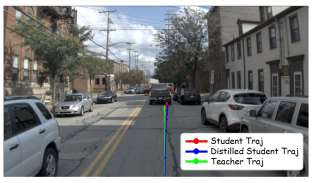
Guidance: 1. **Trajectory Verdict**: The initial trajectory is a problem trajectory that should be corrected before use. Metric summary: PDMS=0.58, NC=1, DAC=1, TTC=0, EP=1, CF=1. 2. **Planning Diagnosis** : **TTC=0** indicates **insufficient time-to-collision margin**; the ego should reduce closing speed or shift away from the conflict path. The longitudinal endpoint is not aligned with the expert-like behavior; the speed profile should be adjusted at a coarse level. 3. **Expert Alignment** : Compared with the expert trajectory, the initial path needs keep the lateral position nearly unchanged, **reduce longitudinal progress moderately** over the planning horizon, and rotate the final heading slightly counter-clockwise. The navigation command is MOVE FORWARD. 4. **Refinement Directive** : First create a safer gap by avoiding the conflict region and **reducing aggressive closing speed**. Starting from the initial trajectory, keep the lateral position nearly unchanged, **reduce longitudinal progress moderately** over the planning horizon. Keep intermediate waypoints smooth and preserve the navigation intent. Primary focus: Scene Interpretation.

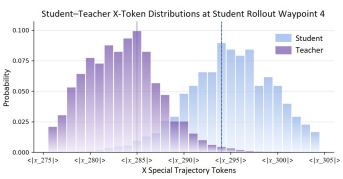
Student: <|x_294|><|y_0|><|theta_1|>, <|x_288|><|y_4|><|theta_2|>, <|x_274|><|y_10|><|theta_2|>, <|x_253|><|y_15|><|theta_2|>, <|x_226|><|y_18|><|theta_2|>, <|x_196|><|y_19|><|theta_1|>, <|x_165|><|y_18|><|theta_1|>, <|x_136|><|y_16|><|theta_1|>

Teacher: <|x_294|><|y_0|><|theta_1|>, <|x_284|><|y_4|><|theta_2|>, <|x_261|><|y_11|><|theta_3|>, <|x_226|><|y_17|><|theta_3|>, <|x_186|><|y_20|><|theta_3|>, <|x_142|><|y_19|><|theta_2|>, <|x_102|><|y_16|><|theta_1|>, <|x_70|><|y_12|><|theta_1|>

Distilled Student: <|x_288|><|y_0|><|theta_1|>, <|x_279|><|y_5|><|theta_2|>, <|x_257|><|y_10|><|theta_2|>, <|x_226|><|y_14|><|theta_2|>, <|x_192|><|y_16|><|theta_2|>, <|x_162|><|y_16|><|theta_1|>, <|x_136|><|y_15|><|theta_1|>, <|x_114|><|y_13|><|theta_1|>

Scenario: At-Fault Collision Risk





Query: <image> is the front view. Command: MOVE FORWARD. Velocity: [7.14, -0.10]. Acceleration: [-0.18, -0.02]. Historical trajectory: [-10.56, -0.07, 0.01], [-6.98, -0.03, 0.01], [-3.47, -0.01, 0.00].\n Plan the action tokens for the ego over the next four seconds, where each group of tokens corresponds to the vehicle's motion within a 0.5-second time window.

Guidance: 1. **Trajectory Verdict** : The initial trajectory is a problem trajectory that should be corrected before use. Metric summary: PDMS=0, NC=0, DAC=1, TTC=0, EP=0, CF=1. 2. **Planning Diagnosis** : **NC=0** indicates an **at-fault collision risk**; the first collision occurs near waypoint 8 around x=21.66m, y=-0.18m with a vehicle. The refinement must first increase clearance from this conflict point. TTC=0 indicates insufficient time-to-collision margin; the ego should reduce closing speed or shift away from the conflict path. 3. **Expert Alignment** : Compared with the expert trajectory, the initial path needs shift the future waypoints slightly left, **reduce longitudinal progress moderately** over the planning horizon, and keep the final heading nearly unchanged. The navigation command is MOVE FORWARD. 4. **Refinement Directive** : First create a safer gap by **avoiding the conflict region** and reducing aggressive closing speed. Starting from the initial trajectory, shift the future waypoints slightly left, **adjust longitudinal progress only after the hard safety constraint is fixed**, and keep the final heading nearly unchanged. Keep intermediate waypoints smooth and preserve the navigation intent. Primary focus: Scene Interpretation.

Student: <|x_343|><|y_0|><|theta_0|>, <|x_334|><|y_m1|><|theta_m1|>, <|x_318|><|y_m3|><|theta_0|>, <|x_294|><|y_m3|><|theta_0|>, <|x_266|><|y_m3|><|theta_0|>, <|x_235|><|y_m3|><|theta_0|>, <|x_203|><|y_m3|><|theta_0|>, <|x_173|><|y_m2|><|theta_0|>

Teacher: <|x_343|><|y_0|><|theta_0|>, <|x_327|><|y_m1|><|theta_0|>, <|x_297|><|y_m1|><|theta_0|>, <|x_254|><|y_0|><|theta_0|>, <|x_208|><|y_0|><|theta_0|>, <|x_165|><|y_m1|><|theta_0|>, <|x_126|><|y_m1|><|theta_0|>, <|x_96|><|y_0|><|theta_0|>

Distilled Student: <|x_343|><|y_0|><|theta_0|>, <|x_327|><|y_m2|><|theta_m1|>, <|x_304|><|y_m3|><|theta_0|>, <|x_263|><|y_m4|><|theta_0|>, <|x_220|><|y_m4|><|theta_0|>, <|x_182|><|y_m3|><|theta_m1|>, <|x_150|><|y_m2|><|theta_0|>, <|x_126|><|y_m2|><|theta_0|>

Figure 4: **Detailed qualitative visualization of Reflect-VLA.** We present three representative failure cases: *Drivable-Area Violation*, *Insufficient TTC Margin*, and *At-Fault Collision Risk*. For each scenario, we show the front-view image, the initial student trajectory, the guidance-conditioned teacher trajectory, and the distilled student trajectory, together with the constructed Reflective Guidance and the corresponding token-level distribution comparison at a critical waypoint under the same student-generated prefix. The results illustrate how Reflective Guidance shifts the teacher toward safer and more expert-aligned trajectory tokens, and how Dual-OPSD transfers these corrective preferences to the distilled student.